Práctica 2
Videoclub distribuido

Base de datos, microservicios, API REST y optimización

Asignatura	Sistemas Informáticos
Curso	2026/2027
Universidad	Escuela Politécnica Superior Universidad Autónoma de Madrid
Proyecto	Videoclub implementado con PostgreSQL, Quart y SQLAlchemy Core asíncrono
Autores	Manuel Salvador Mayor Alejandro González García
Grupo / equipo	Grupo 02 · Equipo 1311
Fecha	20 de octubre de 2026
Entrega	P2.zip

Memoria técnica de la solución implementada.

1. Introducción y objetivos

El objetivo de esta práctica es construir un videoclub distribuido mediante dos microservicios independientes. Los usuarios pueden registrarse, autenticarse, consultar un catálogo de películas, recargar su saldo, reservar películas en un carrito, pagar el carrito y valorar películas. Los administradores pueden gestionar el catálogo y consultar el historial de ventas por año y país.

La solución utiliza PostgreSQL como fuente única de datos, Docker Compose para orquestar los contenedores, Quart como servidor HTTP asíncrono y SQLAlchemy Core con el dialecto asíncrono de PostgreSQL para todas las consultas desde Python.

2. Planificación del desarrollo

Se ha seguido la planificación recomendada en el enunciado. Esta división permite cerrar primero la consistencia de los datos, después exponerla de forma segura mediante los servicios y, finalmente, medir y documentar el rendimiento.

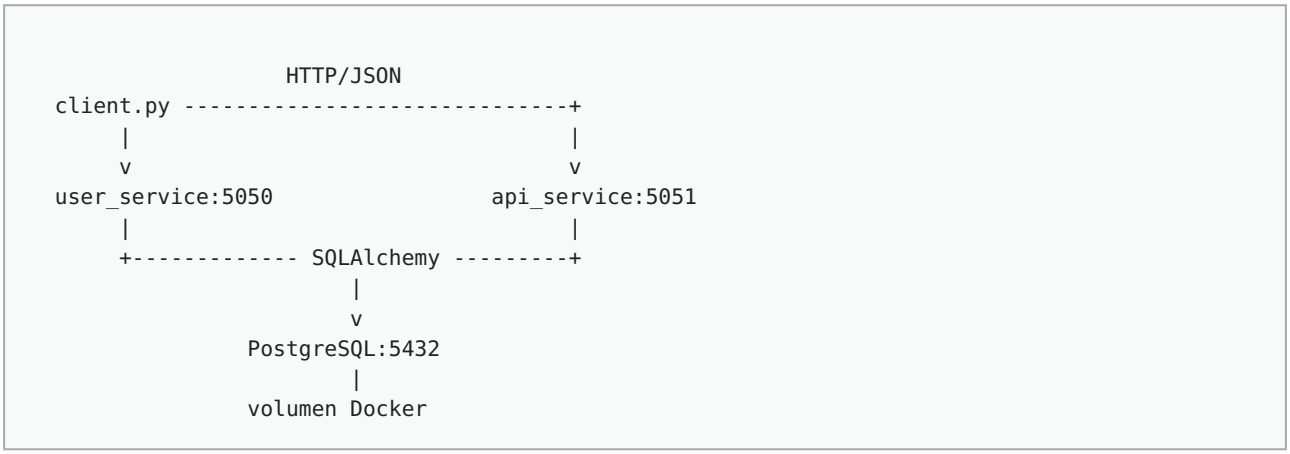
Periodo	Trabajo planificado	Resultado
Semana 1	Diseño de la base de datos, relaciones, restricciones, triggers y procedimientos/funciones almacenadas.	Modelo relacional de usuarios, catálogo, carritos, compras y valoraciones; `schema.sql` y `populate.sql`.
Semana 2	Programación de la API REST y de los dos microservicios.	Servicios Quart asíncronos, autenticación stateless, endpoints de usuarios y videoclub, cliente de pruebas y contenedores.
Resto de días hasta la entrega	Optimización de la consulta, memoria y preparación de la entrega.	`EXPLAIN` antes/después, índices, función optimizada, README, cliente de tolerancia a fallos y esta memoria.

3. Arquitectura de la solución

3.1. Componentes

Componente	Responsabilidad	Puerto
		9999

PostgreSQL 16	Persistencia, restricciones, transacciones, triggers y funciones almacenadas.	
`user_service`	Alta, login, logout, recarga y baja lógica de usuarios.	5050
`api_service`	Catálogo, carritos, pagos, valoraciones e informe de ventas.	5051
`client.py`	Pruebas funcionales y casos de error contra ambos servicios.	-



Los dos servicios se encuentran en la red privada de Compose y acceden al host `database`. Desde el ordenador se publican los puertos indicados para poder ejecutar el cliente y observar las respuestas.

3.2. Persistencia y configuración

PostgreSQL utiliza el volumen `p2_database_data`, montado en `/var/lib/postgresql/data`. Los scripts `database/schema.sql` y `database/populate.sql` se montan en `/docker-entrypoint-initdb.d` y se ejecutan automáticamente la primera vez que se inicializa un volumen vacío.

El secreto de sesión se proporciona mediante `.env` y no se incluye en el repositorio. `.env.example` sirve como plantilla y los dos servicios reciben el mismo valor de `TOKEN_SECRET`.

4. Diseño de la base de datos

4.1. Entidades principales

Tabla	Información	Decisiones relevantes
`users`	Cuenta, país, saldo, rol y estado.	`username` único; saldo no negativo; baja lógica con `active`; `session_version` invalida tokens.
`directors`	Directores del catálogo.	Una película puede conservarse aunque su director se elimine: `ON DELETE SET NULL`.

`actors`	Actores.	Se relacionan con películas mediante una tabla intermedia.
`films`	Título, año, precio, stock y director.	Precio y stock no negativos; `active` permite borrado lógico y conserva el historial.
`carts`	Pedido abierto, pagado o cancelado.	Un único carrito abierto por usuario mediante índice único parcial.
`cart_films`	Líneas, cantidades y precio histórico.	Clave primaria compuesta; `unit_price` evita que el precio actual modifique compras ya iniciadas.
`ratings`	Valoración de usuario y película.	Valor entre 0 y 10; una valoración por usuario y película.
`film_actors`	Relación película-actor.	Resuelve la relación N:M y conserva el personaje interpretado.

4.2. Diagrama entidad-relación

```
USERS 1 -------- N CARTS 1 -------- N CART_FILMS N -------- 1 FILMS
  |                  |                                      |
  |                  |                                      | N
  |                  |                                      |
  |                  |                                      1 DIRECTORS
  |                  |
  |                  +-- estado, total_price, paid_at
  |
  +-- N RATINGS N -- 1 FILMS

FILMS 1 -------- N FILM_ACTORS N -------- 1 ACTORS

Claves: users.user_id, films.film_id, carts.cart_id.
Clave compuesta: cart_films(cart_id, film_id).
Clave compuesta: film_actors(film_id, actor_id).
```

El modelo garantiza la integridad mediante claves foráneas, restricciones CHECK, unicidad y acciones de borrado adecuadas. Las compras se conservan como líneas históricas aunque una película se desactive.

5. Triggers y funciones almacenadas

5.1. Gestión automática del carrito

- `manage_cart_film_stock()`: bloquea el carrito, comprueba que está abierto y descuenta stock al insertar o aumentar una línea. Al borrar o reducir unidades, devuelve el stock.
- `recalculate_cart_total()`: recalcula `carts.total_price` después de cada cambio en `cart_films`.

- `handle_cart_status_change()`: al cambiar a `paid`, comprueba saldo, descuenta el coste y establece `paid_at`. Impide pagar carritos vacíos o pagar dos veces.

5.2. Funciones de negocio

Función	Uso
`get_or_create_open_cart`	Obtiene o crea el carrito abierto de un usuario activo.
`add_film_to_cart`	Aumenta una línea o la crea; los triggers reservan stock y calculan el total.
`remove_film_from_cart`	Elimina una línea y devuelve las unidades reservadas.
`pay_open_cart`	Cambia el estado a pagado y devuelve identificador, total y fecha.
`cancel_open_cart`	Vacía y cancela un carrito abierto, liberando stock.
`average_film_rating`	Calcula la media de las valoraciones y devuelve cero si no hay votos.
`sales_by_year_country`	Devuelve las líneas vendidas para un año y un país.

> En PostgreSQL se han utilizado funciones almacenadas porque las consultas de valoración e informe devuelven datos y pueden invocarse desde un SELECT. Cumplen la finalidad de los procedimientos almacenados solicitados y centralizan la lógica en la base de datos.

6. Implementación de los microservicios

6.1. SQLAlchemy Core asíncrono

La conexión se configura con `create_async_engine` y `AsyncSession`. Todas las rutas que acceden a PostgreSQL son `async def` y ejecutan las sentencias con `await session.execute(text(...), parametros)`.

```
async with async_session() as session:
    async with session.begin():
        result = await session.execute(
            text("SELECT ... WHERE film_id = :film_id"),
            {"film_id": film_id},
        )
```

Las consultas son parametrizadas. No se interpolan datos recibidos por HTTP en el SQL. Las operaciones complejas combinan autenticación y modificación

de datos con CTEs o funciones SQL para respetar el requisito de un único `execute()` por operación.

6.2. Autenticación stateless

Al iniciar sesión se incrementa `session_version`. El token contiene internamente el identificador y la versión, y se cifra con Fernet usando una clave derivada de `TOKEN_SECRET`. El token se envía en `Authorization: Bearer <token>`. No se almacenan tokens. Fernet incorpora la fecha de creación y ambos servicios aplican un TTL de 3600 segundos: al cumplirse una hora, el token se rechaza automáticamente y el usuario debe iniciar sesión de nuevo. Cada endpoint protegido descifra el token y valida en SQL que el usuario sigue activo y que la versión coincide. Login, logout y baja lógica invalidan tokens anteriores incluso antes del TTL.

6.3. Endpoints

Servicio	Método y ruta	Resultado
Usuarios	`PUT /user`	Alta.
Usuarios	`POST /user/login`	Autenticación y token.
Usuarios	`POST /user/logout`	Invalidación de sesión.
Usuarios	`PATCH /user/recharge`	Recarga de saldo.
Usuarios	`DELETE /user`	Baja lógica.
Videoclub	`POST /films`	Alta de película para administradores.
Videoclub	`DELETE /films/<id>`	Baja lógica de película.
Videoclub	`GET /films`	Listado filtrable.
Videoclub	`POST /cart/films`	Añadir al carrito.
Videoclub	`POST /cart/pay`	Pago y fecha.
Videoclub	`PUT /films/<id>/rating`	Voto.
Videoclub	`GET /films/<id>/rating`	Media.
Videoclub	`GET /ventas/<año>/<país>`	Informe de ventas para administradores.

7. Cliente de pruebas y tolerancia a fallos

El fichero `client/client.py` usa `requests` y ofrece el modo `--examples`. Cada petición muestra el cuerpo y compara el código HTTP recibido con el código

esperado. Los errores de red se capturan y se informa del problema sin abortar la batería completa.

Área	Casos correctos	Casos de error comprobados
Usuarios	Alta, login, recarga, logout y baja.	Duplicado, credenciales erróneas, campos vacíos, recarga negativa y petición sin token.
Catálogo	Listado por título, año, director y actor.	Año no numérico, creación sin permisos y sin token.
Carrito	Añadir unidades y pagar.	Película inexistente, cantidad cero, pago sin carrito.
Valoraciones	Crear, modificar y consultar media.	Valor fuera de 0-10 y película inexistente.
Ventas	Consulta de administrador.	Consulta de usuario normal.
Sesión	Logout y nueva autenticación.	Reutilización de token invalidado y login de usuario dado de baja.

```
cd p2
cp .env.example .env
docker-compose up -d --build
python -m pip install -r requirements.txt
python client/client.py --examples
```

La ejecución completa terminó sin marcas FALL0. Las capturas siguientes muestran los contenedores activos y la batería de casos correctos y erróneos, incluidos autenticación, permisos, saldo insuficiente, pago, valoraciones, ventas e invalidación de tokens.

7.1. Evidencias de ejecución



Figura 1. Los tres contenedores activos y PostgreSQL en estado healthy.

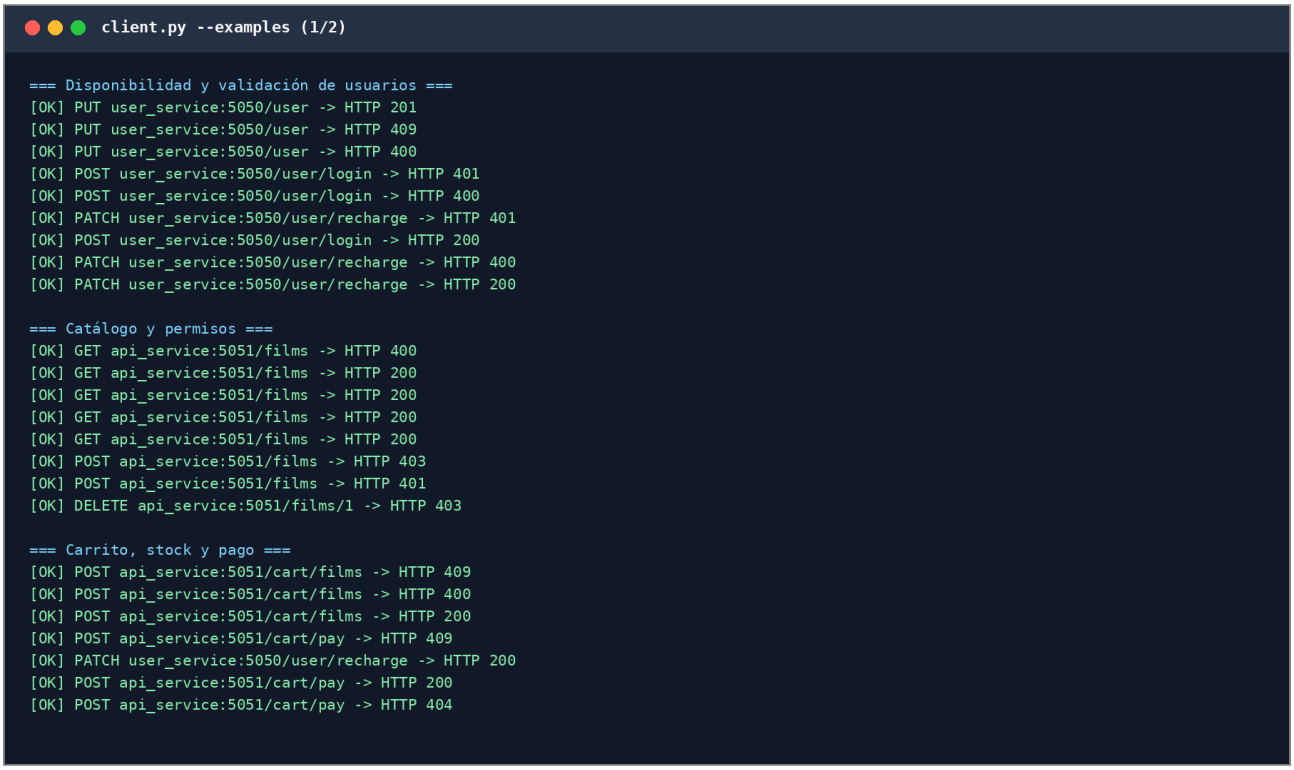


Figura 2. Usuarios, catálogo, permisos, carrito y pago.

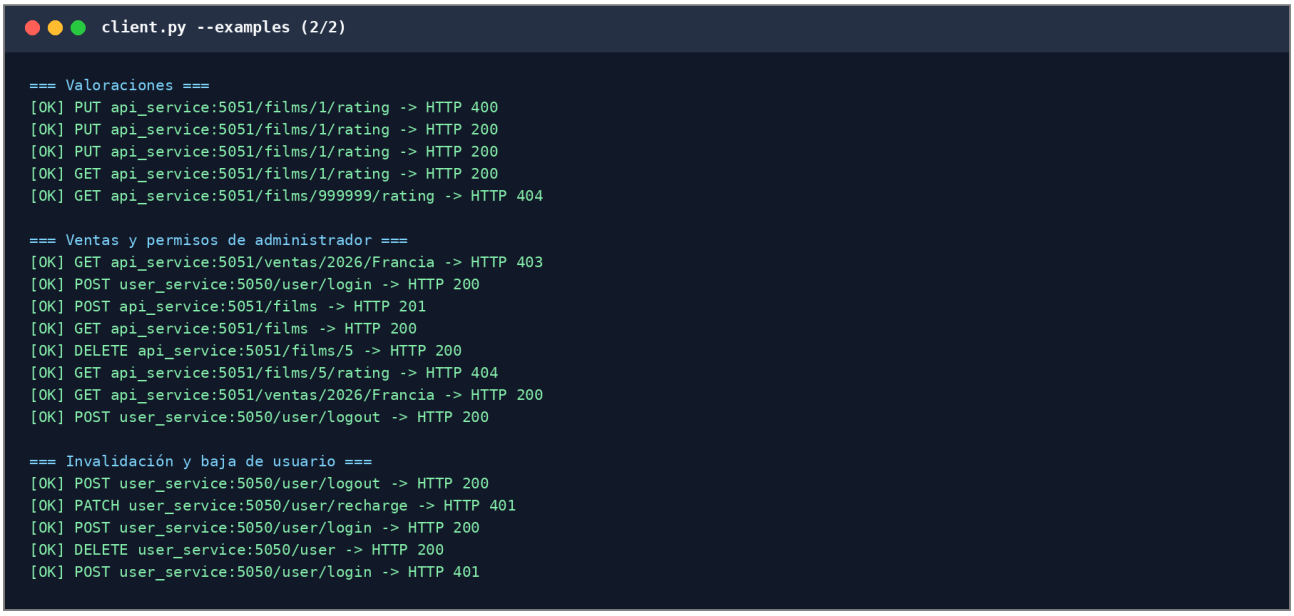


Figura 3. Valoraciones, ventas, logout y baja lógica.

8. Optimización de ventas

8.1. Problema inicial

El informe une `carts`, `users`, `cart_films` y `films`, filtra por compras pagadas, año de pago y país, y ordena por fecha, carrito y película. La versión inicial aplicaba `EXTRACT(YEAR FROM paid_at)`. Al aplicar una función sobre la

columna, un índice normal sobre `paid_at` no puede utilizarse de forma eficaz para localizar directamente el intervalo.

8.2. Estrategia

1. Ejecutar el plan base con `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)`.
2. Crear el índice parcial `carts(paid_at, user_id, cart_id) WHERE status = 'paid'`. Solo almacena las filas que puede devolver el informe.
3. Crear el índice funcional `users(LOWER(country), user_id)`, coherente con el filtro insensible a mayúsculas.
4. Sustituir el filtro del año por `paid_at >= inicio` y `paid_at < fin`. Este predicado permite un index scan o bitmap index scan según el tamaño de la tabla y la distribución.
5. Repetir el mismo EXPLAIN y comparar coste estimado, tiempo real, buffers leídos y nodos de acceso.

Todas las sentencias están en `database/optimizacion.sql`. El script también reemplaza `sales_by_year_country()` para que la API aplique la misma estrategia, no solo el ejemplo de laboratorio.

```
docker exec -i si_p2_database psql -U alumnodb -d si1 \
  < database/optimizacion.sql
```

8.3. Resultados medidos

Para evitar que el tamaño mínimo de `populate.sql` ocultase el efecto de los índices, se midió con 5.003 usuarios, 50.002 carritos y 50.003 líneas. La consulta analizada fue ventas de 2026 para clientes de Francia.

Métrica	Plan original	Plan optimizado
Acceso a users/carts	Seq Scan / Seq Scan	Bitmap Index Scan / Bitmap Index Scan
Buffers	shared hit=2017	shared hit=1641, read=25
Planning Time	1,963 ms	3,290 ms
Execution Time	11,614 ms	4,965 ms

El tiempo de ejecución se redujo aproximadamente un 57,25 %. El ligero aumento del tiempo de planificación queda compensado por la reducción del recorrido de datos. Con las pocas filas iniciales PostgreSQL puede preferir un recorrido secuencial; el beneficio aparece al crecer el histórico.

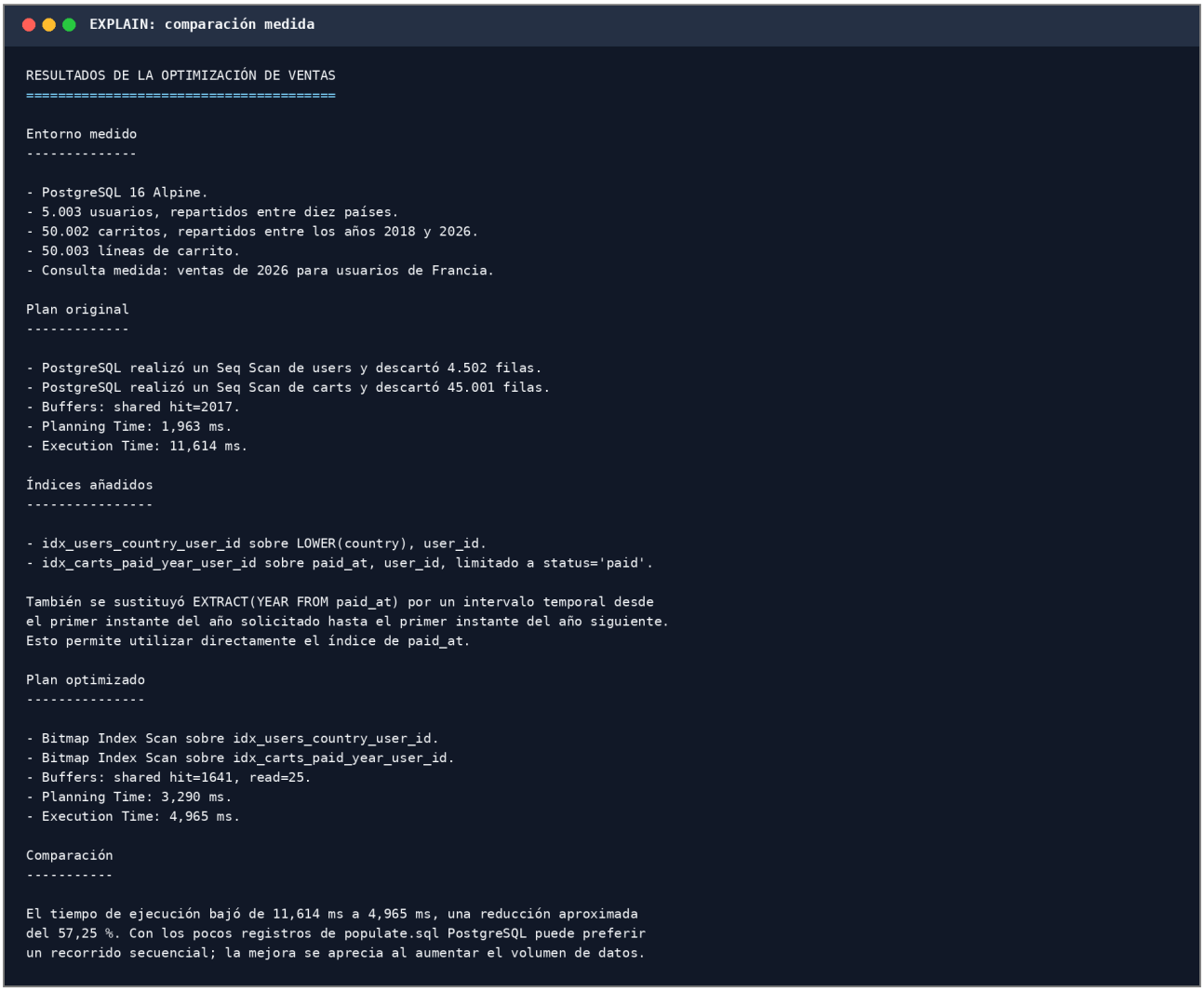


Figura 4. Resumen de las salidas reales de EXPLAIN (ANALYZE, BUFFERS).

9. Seguridad, consistencia y decisiones

- Las contraseñas no se almacenan en claro; se guarda su hash SHA-256.
- Los tokens son Fernet, caducan después de una hora y el secreto se configura fuera del repositorio.
- Los valores HTTP se envían como parámetros de SQLAlchemy, evitando inyección SQL.
- Los pagos se realizan dentro de una transacción y el descuento verifica saldo suficiente.
- Los triggers serializan el acceso al carrito y actualizan stock y total en la base de datos, evitando inconsistencias entre clientes.
- Las películas y usuarios se desactivan lógicamente para conservar el historial de operaciones.
- Las restricciones de PostgreSQL actúan como última barrera de integridad, incluso si otro cliente accede directamente a la base de datos.

10. Organización de la entrega

La entrega debe ser un único archivo `P2.zip` con los fuentes, scripts SQL, dependencias, Docker Compose, cliente y esta memoria en PDF. No deben incluirse entornos virtuales, cachés, bytecode ni metadatos del sistema operativo.

```
find . -type d \( -name __pycache__ -o -name .venv -o -name venv \) \
  -prune -exec rm -rf {} +
find . -type f \( -name '*.pyc' -o -name '*.pyo' \) -delete
zip -r P2.zip . -x '*.git/*' '*.env' '*.venv/*' '*__pycache__/*'
```

La validación final se realizó desde un volumen limpio. Se construyeron los contenedores, se ejecutó el cliente sin fallos, se verificó la persistencia después de reiniciar PostgreSQL y se ejecutó íntegramente `optimizacion.sql`. Las evidencias se incluyen en los apartados 7 y 8.

11. Conclusiones

La implementación separa claramente la gestión de usuarios de la lógica del videoclub, manteniendo una única base de datos transaccional. La lógica crítica de stock, totales y pagos reside en PostgreSQL mediante triggers y funciones, mientras que Quart expone una interfaz REST asíncrona y parametrizada. El cliente automatiza tanto los casos normales como los errores y el script de optimización deja reproducible la comparación del informe de ventas.